BCSE307P Compiler Design Lab

Innovative Project

MiniLang: A Complete Compiler and Stack Virtual Machine with an Interactive Learning Playground

by

Reg No	Team Member Names
24BDS0255	Ishu Raj
24BDS0284	Parth Bagwe
24BCE2847	Siddiqa
24BDS0203	Tejas Sinha

Under the Supervision of

Prof. MUTHUNAGAI S U

Assistant Professor Sr. Grade-2

School of Computer Science Engineering and Information Systems

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

November 2026

1. Abstract:

Compiler design is usually taught one phase at a time, on paper, while the compilers students actually use hide every intermediate step behind a single command. This project, **MiniLang**, closes that gap. MiniLang is a small imperative programming language with whole numbers, lists, functions, recursion, loops and input, together with a complete compiler for it, written from scratch in Python 3 using only the standard library and no parser or lexer generators.

A program passes through every classical phase: a lexer; a recursive-descent parser; semantic analysis with symbol tables; a static type checker; an intermediate representation of three-address code organised into basic blocks and a control-flow graph; data-flow analyses (constant propagation, liveness and definite assignment); an optimizer (constant folding, dead-store elimination, peephole optimisation and tail-call optimisation); a code generator; and a stack-based virtual machine with an instruction set of 30 opcodes. The front end recovers from errors, so a single run reports every mistake with its line number and a helpful hint.

A Flask web application makes each phase visible: a playground with a tab for every stage and a step-through debugger, a game of 21 graded levels, 12 lessons that teach the language, and 8 chapters that explain the compiler using its own source code. The implementation is verified by 339 automated unit tests, which run on every push through GitHub Actions.

Keywords: compiler design, lexical analysis, recursive-descent parsing, semantic analysis, type inference, data-flow analysis, code optimisation, stack virtual machine, educational software.

2. Introduction:

2.1 Background

A compiler translates a program written in a source language into an equivalent program in a target language. The standard textbook treatment (Aho, Lam, Sethi and Ullman, *Compilers: Principles, Techniques, and Tools*) divides the work into a **front end** (lexical, syntax and semantic analysis), a **middle end** (intermediate code generation and machine-independent optimisation) and a **back end** (code generation). Each phase consumes the output of the one before it: characters become tokens, tokens become a syntax tree, the tree is checked and lowered to intermediate code, the intermediate code is optimised, and finally executable code is produced.

In a laboratory course these phases are usually met separately: a lexer is generated with Lex, a parser with Yacc, and optimisation algorithms are traced by hand on small examples. Production compilers do not help much either, because they show only the final output or an error message. Students therefore rarely see how the phases fit together, or what each phase's data structure looks like for a real program.

2.2 Problem statement

To design and implement a complete compiler for a small but non-trivial programming language, in which every phase is written by hand, is observable and can be explored interactively, so that the compiler works both as a real tool and as a teaching aid for compiler design.

2.3 Objectives

- Define a small imperative language, MiniLang, with a formal context-free grammar.
- Implement every phase from scratch, without generator tools: lexer, parser, semantic analyser, type checker, intermediate code generator, optimizer, code generator and virtual machine.
- Implement the textbook data-flow analyses on a control-flow graph, and use them both for warnings and for optimisation.
- Report errors precisely and recover from them, so that one run shows every mistake.
- Build a web interface that shows the output of every phase and lets the user step through execution.
- Verify correctness with an automated test suite, including tests that the optimizer never changes what a program does.

2.4 The MiniLang language

MiniLang is a dynamically typed language with two kinds of value, whole numbers and lists. Its syntax borrows from C and Python. Table 1 summarises the language.

Table 1: MiniLang at a glance.

Feature	Syntax	Example
Comments	`#` to the end of the line	`# running total`
Variables	`name = expr;` and `+= -= *= /= %=`	`total += x;`
Arithmetic	`+ - * / %` and unary `-`	`(a + b) * 2`
Comparison and logic	`== != < > <= >=`, `and or not`	`x > 0 and not done`
Output and input	`print expr;`, `input name;`	`input n;`

Feature	Syntax	Example
Conditionals	`if … else if … else`	`if x > 0 { print x; }`
Loops	`while`, C-style `for`, `break`, `continue`	`for i = 0; i < n; i += 1 { … }`
Functions	`func name(params) { … }`, `return`, recursion	`func fact(n) { … }`
Lists	literals, indexing, `len()`, `append()`	`xs = [1, 2]; append(xs, 3);`

Listing 1: A MiniLang program that prints the first factorials.

```
func fact(n) {
    if n <= 1 { return 1; }
    return n * fact(n - 1);
}

input count;                  # read a number
for i = 1; i <= count; i += 1 {
    print fact(i);
}
```

3. Novelty in Project:

Small compilers for teaching are common. What distinguishes MiniLang is that it implements the whole textbook pipeline, including the middle end that student projects usually leave out, and makes every intermediate result visible and interactive. The main contributions are:

- **The whole pipeline, observable end to end.** Tokens, the syntax tree, symbol tables, inferred types, three-address code with its control-flow graph, bytecode before and after optimisation, and an execution trace are shown side by side for the same program, linked by line numbers.
- **A real middle end.** Constant propagation (Kildall's iterative algorithm), liveness analysis and definite-assignment analysis run on the control-flow graph. Their results drive both warnings, such as dead stores and possibly undefined variables, and optimisations, and they are drawn on the graph in the interface.
- **Type inference as a data-flow problem.** The type checker treats a type as a set drawn from {number, list}, iterates to a fixed point across all functions, and reports only errors that are certain on every path, so it never rejects a correct program.
- **Error recovery with helpful hints.** The lexer skips invalid characters, the parser uses phrase-level recovery (inserting a missing semicolon) and panic-mode recovery, and semantic analysis keeps going, so one run reports every error. Messages suggest corrections for misspelt keywords and names ("did you mean 'while'?") and for syntax borrowed from other languages, such as `&&` instead of `and`.

- **Optimisations proven safe by testing.** Tests run tricky programs with the optimizer on and off and require identical output, or the identical error on the same line. Tail-call optimisation lets tail recursion 5,000 calls deep run in constant stack space, where the unoptimised program exceeds the call depth limit.
- **A step-through debugger that runs backwards.** The virtual machine records every instruction it executes, so the user can step forwards and backwards and inspect the operand stack and every call frame at each step.
- **Learning by playing.** Twenty-one levels, 13 programming challenges and 8 "bug hunts" (one for each kind of error), are checked against hidden tests. Stars are awarded from the compiler's own metrics, optimised bytecode size and virtual-machine steps, so players learn why one solution is faster than another.
- **Documentation that cannot go stale.** The 8 compiler chapters quote the compiler's source directly from its files using Python's `ast` module, and the examples in the 12 language lessons are compiled and run when the page is rendered. A test fails if a quote or an example no longer matches the code.
- **No dependencies, works offline.** The compiler uses only the Python standard library; Flask is needed for the website alone, and the website serves all of its assets itself.

Table 2 compares MiniLang with tools commonly used to explore compilers and programs.

Table 2: Comparison with existing tools.

Tool	What it shows	What it does not show
Compiler Explorer	Assembly, and some intermediate dumps, from production compilers	Execution; industrial compilers' output is too large to follow phase by phase
AST Explorer	The syntax tree produced by many existing parsers	Semantic analysis, intermediate code, optimisation or execution
Python Tutor	Step-by-step execution with variables and call frames	Any compilation phase
Lex / Yacc exercises	Generated lexers and parsers	The middle and back end; the generated code is hard to read
MiniLang	Every phase from characters to an execution trace, for the same program	Scope is deliberately small: a teaching language, not a production one

4. Proposed Work:

4.1 System architecture

The system has two layers (Figure 1). The **compiler** is a pipeline of independent modules: each phase is one Python file that consumes the previous phase's output and produces a well-defined data structure. Any phase can report errors, and every error records its phase

and source line. The **web application** (`api.py` and `app.py`) runs the same pipeline and returns every phase's output as JSON, which the browser renders. The browser never compiles anything itself.

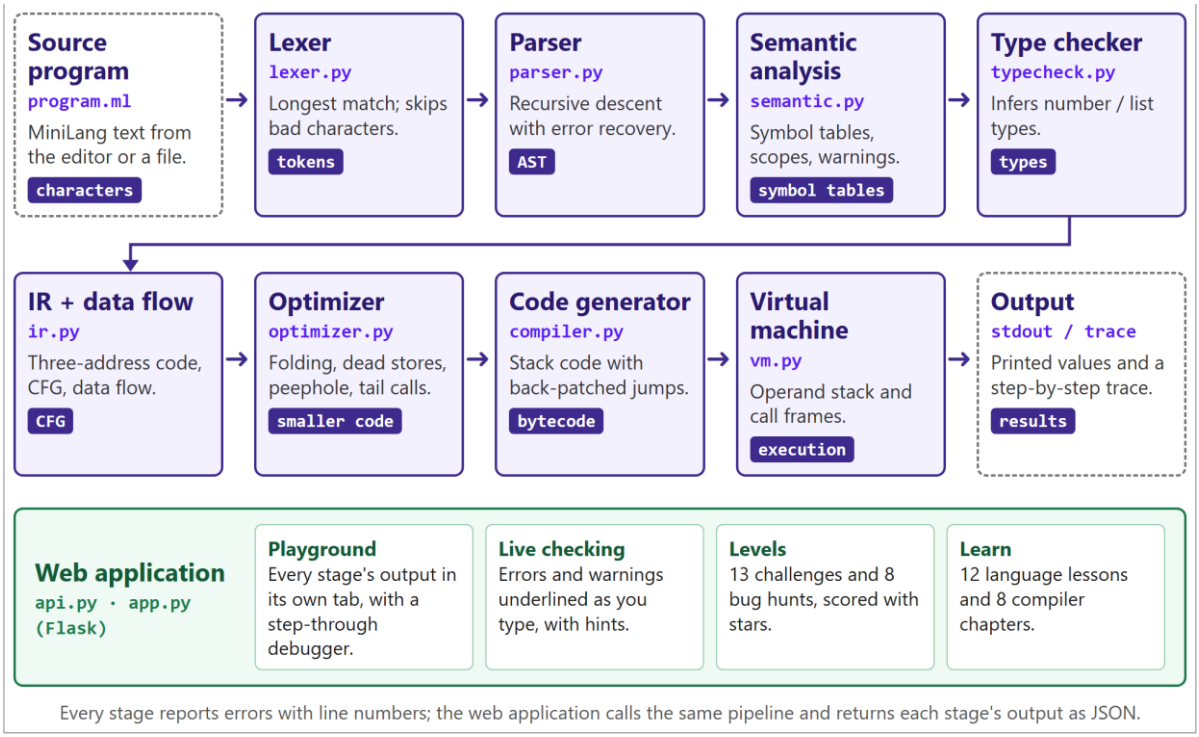


Figure 1: System architecture: the compiler pipeline and the web application.

4.2 Grammar

The syntax is defined by a context-free grammar (Listing 2) designed so that one token of lookahead, or two for a function call, always decides which rule applies. This makes it suitable for recursive descent. Operator precedence is encoded by **layering**: each precedence level is a separate rule that parses its operands with the next, tighter level, and a loop within each rule makes the operators left-associative.

Listing 2: MiniLang grammar, from lowest to highest precedence.

```
program    := (funcdef | statement)* EOF
funcdef    := "func" IDENT "(" (IDENT ("," IDENT)*)? ")" block
statement  := assignment ";" | "print" expr ";" | "input" IDENT ";" | call ";"
            | "if" expr block ("else" (if | block))?
            | "while" expr block
            | "for" assignment? ";" expr? ";" assignment? block
            | "return" expr? ";" | "break" ";" | "continue" ";" | block
assignment := IDENT ("[" expr "]")* ("=" | "+=" | "-=" | "*=" | "/=" | "%=") expr
block      := "{" statement* "}"
expr       := and_expr ("or" and_expr)*
and_expr   := not_expr ("and" not_expr)*
not_expr   := "not" not_expr | comparison
comparison := additive (("=="|"!="|"<"|">"|"<="|">=") additive)?
additive   := term (("+"|"-") term)*
term       := unary (("*"|"/"|"%") unary)*
unary      := "-" unary | postfix
postfix    := primary ("[" expr "]")*
primary    := INT | call | IDENT | "(" expr ")" | "[" (expr ("," expr)*)? "]"
call       := IDENT "(" (expr ("," expr)*)? ")"
```

4.3 Design of each phase

a) Lexical analysis (lexer.py)

The lexer scans the source one character at a time with one character of lookahead and applies the **longest-match** rule, so `<=` is read as one operator rather than `<` followed by `=`. Words are checked against the keyword table to separate keywords from identifiers. Each token records its type (`INT`, `IDENT`, `KEYWORD`, `OP` or `EOF`), its value and its line number. An unexpected character is reported and skipped, so lexing continues and every bad character is found in one run.

b) Syntax analysis (parser.py)

A hand-written **recursive-descent parser** has one method per grammar rule and builds an abstract syntax tree (AST) whose nodes keep their source line. Compound assignments such as `x += e` are desugared to `x = x + e` here, so later phases never see them. On an error the parser either inserts a missing `;` at the end of a line (**phrase-level recovery**) or skips to the start of the next statement (**panic-mode recovery**). It keeps at most one error per line and stops after 20.

c) Semantic analysis (semantic.py)

Two passes over the AST build the **symbol tables**. The first collects every function, so a function may be called before it is defined; the second resolves every name in its scope (global, or local to a function). It reports undefined variables and functions, wrong argument counts, `break` or `continue` outside a loop and `return` outside a function. It also warns about unused variables, unreachable code, dead stores, possibly undefined variables

and the **scope trap**, where assigning to a global's name inside a function silently creates a local variable instead.

d) Type checking (typecheck.py)

MiniLang is dynamically typed, but many type errors can still be found before the program runs. The checker infers, for every variable at every point, the **set of types** it may hold, using a forward data-flow analysis over the control-flow graph whose join is set union. Parameter and return types are found by iterating over all the functions until nothing changes (an interprocedural fixed point). An error is reported only if an operation is wrong for every possible type, for example adding a number to a value that is certainly a list.

e) Intermediate representation and data-flow analysis (ir.py)

The AST is lowered to **three-address code**, in which every instruction has at most one operator, using compiler-generated temporaries (`t1`, `t2`, …). The code is divided into **basic blocks** at leaders (the first instruction, every label and every instruction after a jump), and the blocks are joined into a **control-flow graph** (CFG). Three analyses run on the graph:

- **Constant propagation** (forward, Kildall's algorithm): a variable is constant at a block's entry only if every predecessor agrees on its value. Branches whose condition is constant prune the edges that can never be taken.
- **Liveness** (backward): live_in(B) = use(B) ∪ (live_out(B) − def(B)), where live_out(B) is the union of live_in over B's successors. A store to a variable that is not live afterwards is a **dead store**.
- **Definite assignment** (forward, a *must* analysis with intersection at joins): finds reads that may happen before any assignment on some path.

f) Optimisation (optimizer.py)

The optimizer runs four passes. **Constant propagation** replaces reads of proven constants with their values. **Constant folding** evaluates constant expressions at compile time, but never one that would fail, such as a division by zero, so the runtime error is kept. **Dead-store elimination** removes stores whose value is never read, provided computing the value cannot fail. A **peephole pass** removes branches with constant conditions, threads jumps that lead to other jumps, deletes unreachable code, and turns a `CALL` immediately followed by `RET` into `TAIL_CALL`, which reuses the current frame.

g) Code generation (compiler.py)

The code generator walks the AST in **post-order**, emitting instructions for a stack machine: the operands are pushed before their operator is applied. Forward jumps for `if`, `while`, `for`,

`break`, `continue` and function calls are emitted with a placeholder target and **back-patched** once the target address is known. `and` and `or` are compiled with short-circuit jumps. The program is laid out as the top-level code, a `HALT` instruction, and then each function body.

h) Virtual machine (vm.py)

The virtual machine runs a **fetch-decode-execute** loop over the bytecode, with an operand stack and a call stack of frames; each frame holds a function's local variables and its return address. It checks the types of values at run time and enforces limits on steps, printed lines and call depth, so that no program, including one sent from a browser, can run forever. A hook called after every instruction records the trace used by the step-through debugger. Table 3 lists the instruction set.

Table 3: The 30 instructions of the MiniLang virtual machine.

Group	Instructions
Stack and variables	`PUSH`, `LOAD`, `STORE`, `POP`
Arithmetic	`ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `NEG`
Comparison and logic	`EQ`, `NE`, `LT`, `GT`, `LE`, `GE`, `NOT`
Control flow	`JUMP`, `JUMP_IF_FALSE`, `CALL`, `TAIL_CALL`, `RET`, `HALT`
Lists	`BUILD_LIST`, `INDEX`, `STORE_INDEX`, `LEN`, `APPEND`
Input and output	`PRINT`, `INPUT`

4.4 Web application

The website is a Flask application in three parts: a **landing page** that introduces the project with a live self-test and an animated stack machine; the **playground**, where code is written and every phase is shown; and the **learning section**, with lessons on the language and chapters on the compiler. Table 4 lists its routes.

Table 4: Routes of the web application.

Route	Method	Purpose
`/`	GET	Landing page
`/play`	GET	Playground; `?example=`, `?code=` and `#level=` open a program or a level directly
`/learn`, `/learn/language/<lesson>`, `/learn/<chapter>`	GET	Learning section: lessons and chapters
`/run`	POST	Compile and run a program; returns every phase's output and the execution trace as JSON
`/lint`	POST	Static checks only, used for live error checking while typing

Route	Method	Purpose
`/check`	POST	Run a level's hidden tests and award stars
`/stage`	POST	Run a snippet up to one phase, for the demos in the chapters

4.5 Testing strategy

Every phase has its own unit tests, written with Python's `unittest` module. On top of these, **behaviour-preservation tests** run a list of tricky programs with the optimizer on and off and require the same output, or the same error on the same line. **Content tests** check that every level's reference solution earns three stars, every lesson example produces the output its lesson describes, and every source quote in the chapters still points at real code. A GitHub Actions workflow runs the whole suite on every push.

5. Implementation & Screenshots:

5.1 Tools and environment

Table 5: Tools and environment.

Component	Technology
Language	Python 3 (developed and tested on Python 3.13); the compiler uses only the standard library
Web framework	Flask 3
Front end	HTML, CSS and JavaScript with no framework; CodeMirror 5 for the code editor
Testing	Python `unittest`; GitHub Actions for continuous integration
Version control	Git and GitHub

5.2 Code organisation

Each phase lives in its own module, so a phase can be read, tested and explained on its own. Table 6 lists the modules; in total the project has about 4,650 lines of Python, plus 2,450 lines of tests.

Table 6: Python modules.

Module	Lines	Responsibility
`lexer.py`	140	Characters to tokens; skips and reports bad characters
`parser.py`	409	Recursive-descent parser, error recovery, keyword hints, `+=` desugaring
`ast_nodes.py`	152	Syntax tree node classes; text and JSON dumps
`semantic.py`	425	Symbol tables, scope checks and warnings

Module	Lines	Responsibility
`typecheck.py`	286	Type inference over the CFG; certain type errors
`ir.py`	782	Three-address code, basic blocks, CFG, data-flow analyses
`optimizer.py`	297	Propagation, folding, dead stores, peephole, tail calls
`compiler.py`	330	Bytecode generation with back-patching; disassembler
`vm.py`	247	Stack virtual machine, limits and tracing
`errors.py`	51	One error class per phase, each with a line number
`main.py`	109	Command line: `--debug`, `--trace`, `--no-opt`, `--input`
`api.py`	214	Runs the pipeline and returns every phase as JSON
`app.py`	273	Flask routes for the website
`levels.py`	491	The 21 levels, hidden tests and star scoring
`guide.py`, `learn.py`	451	Language lessons; compiler chapters, source quoting and demos

5.3 One program through every phase

The following output was produced by the command-line driver (`python main.py example.ml --debug`) for a four-line program. It shows the data structure each phase hands to the next.

Listing 3: Input program.

```
x = 6 * 7;
if x > 40 {
    print x;
}
```

Listing 4: Lexer output (16 tokens, abbreviated).

```
Token(IDENT, 'x', line=1)      Token(OP, '=', line=1)
Token(INT, 6, line=1)          Token(OP, '*', line=1)
Token(INT, 7, line=1)          Token(OP, ';', line=1)
Token(KEYWORD, 'if', line=2)   Token(IDENT, 'x', line=2)
Token(OP, '>', line=2)         Token(INT, 40, line=2)
...                            Token(EOF, None, line=5)
```

Listing 5: Parser output, the abstract syntax tree (condensed).

```
Program  [line 1]
  statements:
    Assign(name='x')  [line 1]
      value:
        BinOp(op='*')  [line 1]
          left:  Number(value=6)
          right: Number(value=7)
    If(else_body=None)  [line 2]
      condition:
        BinOp(op='>')  [line 2]
          left:  Var(name='x')
          right: Number(value=40)
      then_body:
        Block  [line 2]
          statements:
            Print  [line 3]
              value: Var(name='x')
```

Listing 6: Semantic analysis (symbol table) and type checker output.

```
global scope:
  x             global     assigned 1             read 2, 3
globals:
  x             number
```

Listing 7: Three-address code, basic blocks, constants and liveness.

```
3 basic blocks, 2 constant variable reads, 0 unreachable blocks
procedure main
  B0  -> B1, B2
      live in: -   live out: x
      x = 6 * 7
      t1 = x > 40
      if_false t1 goto L1
  B1  -> B2  constants in: x=42
      live in: x   live out: -
      print x
  B2 (L1)  -> exit  constants in: x=42
      live in: -   live out: -
```

Listing 8: Bytecode before and after optimisation.

```
Before optimising (11):           After optimising (3):
0000  PUSH          6               0000  PUSH          42
0001  PUSH          7               0001  PRINT
0002  MUL                           0002  HALT
0003  STORE         x
0004  LOAD          x
0005  PUSH          40
0006  GT
0007  JUMP_IF_FALSE 10
0008  LOAD          x
0009  PRINT
0010  HALT
```

Constant propagation proves that `x` is 42 in every block, so `x > 40` is always true. Folding and the peephole pass then remove the comparison and the branch, and dead-store elimination removes the store to `x`, because after substitution nothing reads it. The virtual machine runs the three remaining instructions and prints `42`.

5.4 Execution trace

With `--trace`, the virtual machine prints every instruction it executes with the operand stack after it. Listing 9 shows the start of the trace for `print fact(5);` using the recursive factorial from Listing 1: each `CALL` opens a new frame, visible as the growing call depth.

Listing 9: The start of an execution trace.

```
[0000] PUSH          5       stack=[5]           ; line 5
[0001] CALL          4       stack=[5]           ; line 5  in fact(n) (depth 1)
[0004] STORE         n       stack=[]            ; line 1  in fact(n) (depth 1)
[0005] LOAD          n       stack=[5]           ; line 2  in fact(n) (depth 1)
[0006] PUSH          1       stack=[5, 1]        ; line 2  in fact(n) (depth 1)
[0007] LE                    stack=[0]           ; line 2  in fact(n) (depth 1)
[0008] JUMP_IF_FALSE 11      stack=[]            ; line 2  in fact(n) (depth 1)
[0011] LOAD          n       stack=[5]           ; line 3  in fact(n) (depth 1)
[0012] LOAD          n       stack=[5, 5]        ; line 3  in fact(n) (depth 1)
[0013] PUSH          1       stack=[5, 5, 1]     ; line 3  in fact(n) (depth 1)
[0014] SUB                   stack=[5, 4]        ; line 3  in fact(n) (depth 1)
[0015] CALL          4       stack=[5, 4]        ; line 3  in fact(n) (depth 2)
...
```

5.5 Error reporting

Each phase reports errors with the phase name, the line and, where possible, a hint. Because the front end recovers, all the errors of one phase are reported together. Listing 10 shows the messages for four small faulty programs.

Listing 10: Error messages from the lexer, parser, semantic analysis and type checker.

```
x = 1
print x;
if x > 0 { print 2 }
  -> ParseError at line 1: expected ';' after assignment, found 'print'
  -> ParseError at line 3: expected ';' after print, found '}'

x = 5 && 3;
  -> LexError at line 1: unexpected characters '&&' (MiniLang writes 'and' instead of '&&')

count = 1;
print cont;
  -> SemanticError at line 2: undefined variable 'cont' (did you mean 'count'?)

xs = [1, 2];
print xs + 1;
n = 5;
print n[0];
  -> TypeError at line 2: '+' needs two numbers, but 'xs' is a list
  -> TypeError at line 4: 'n' is a number, so it can't be indexed with [ ]
```

5.6 Screenshots

Figures 2 to 13 show the web application. The playground's right-hand panel has one tab per phase; each tab shows that phase's output for the program in the editor on the left.

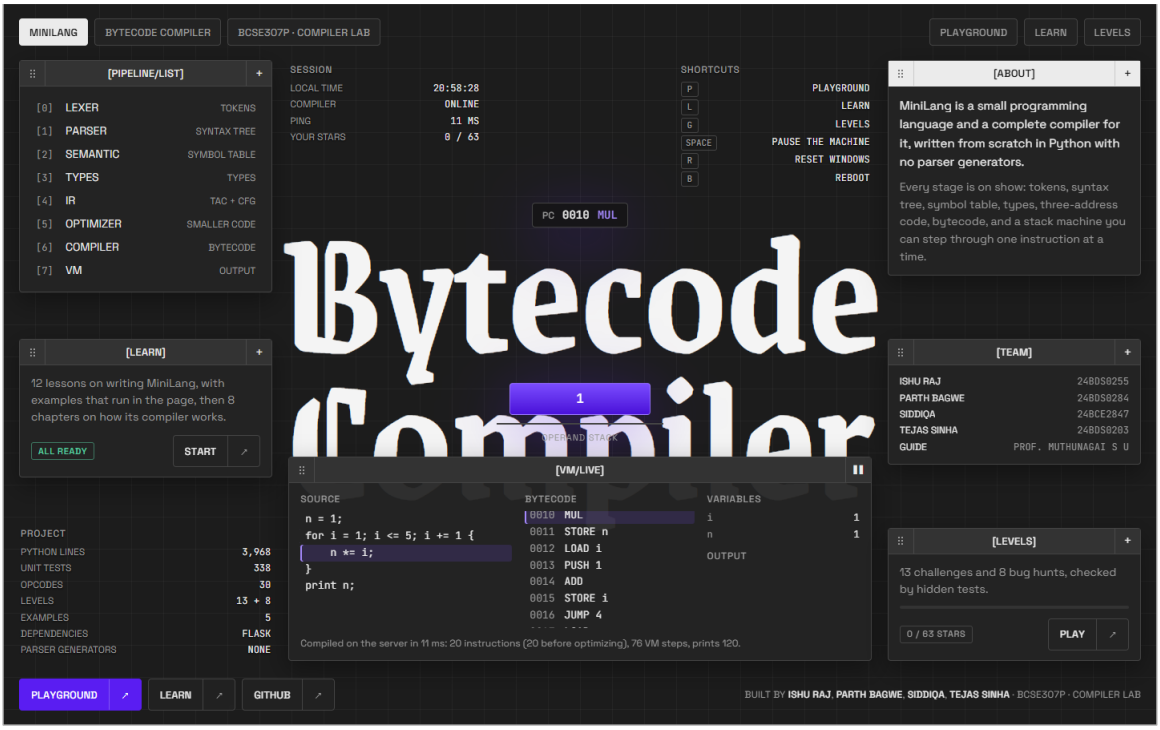


Figure 2: Landing page. The stack machine in the centre replays a real virtual-machine trace.

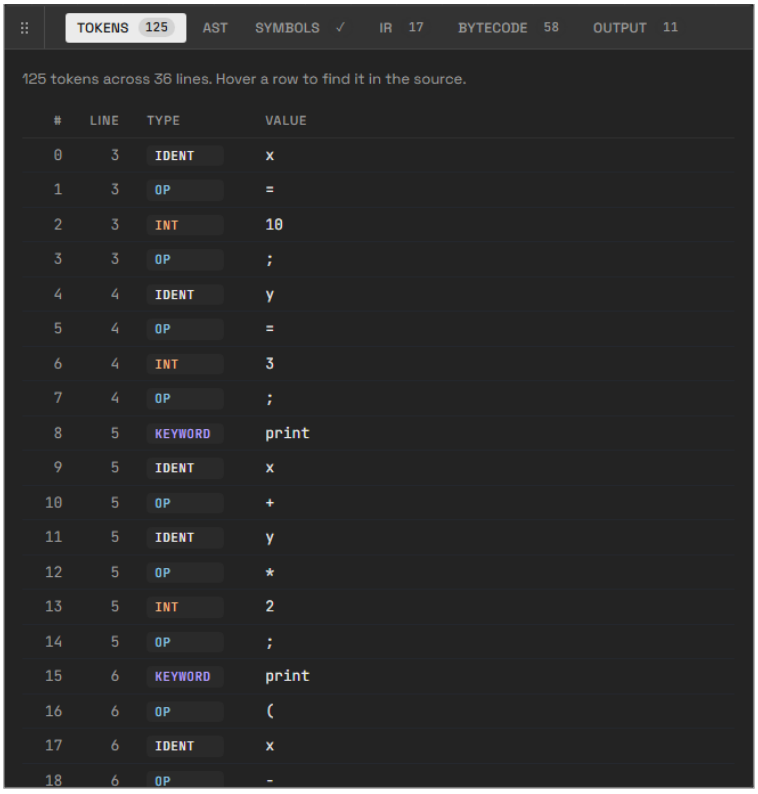


Figure 3: Playground, Tokens tab: the lexer's output.

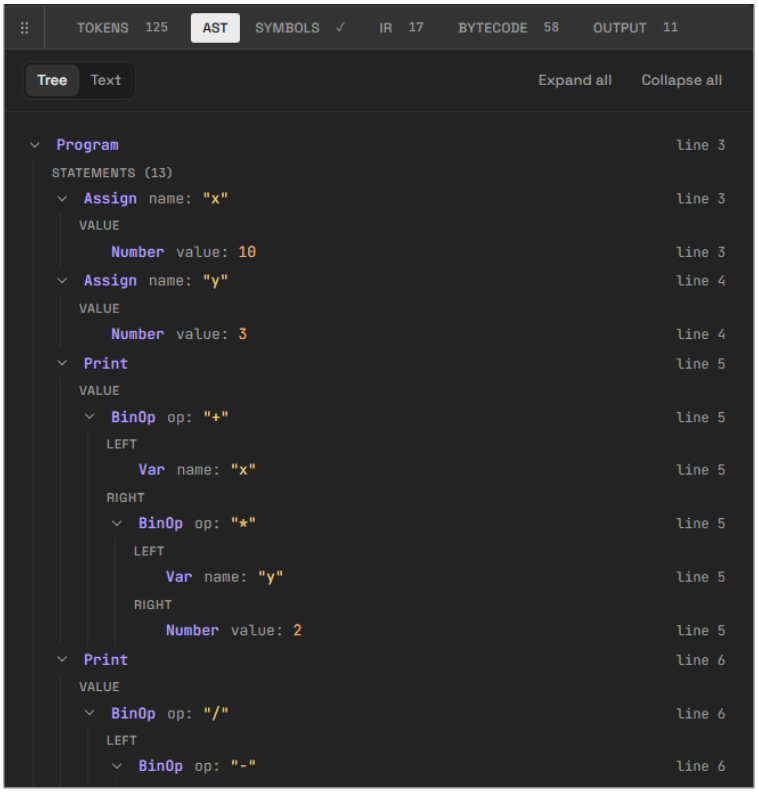


Figure 4: Playground, AST tab: the syntax tree, with the line of every node.



Figure 5: Playground, Symbols tab: symbol tables per scope, with inferred types and function signatures.

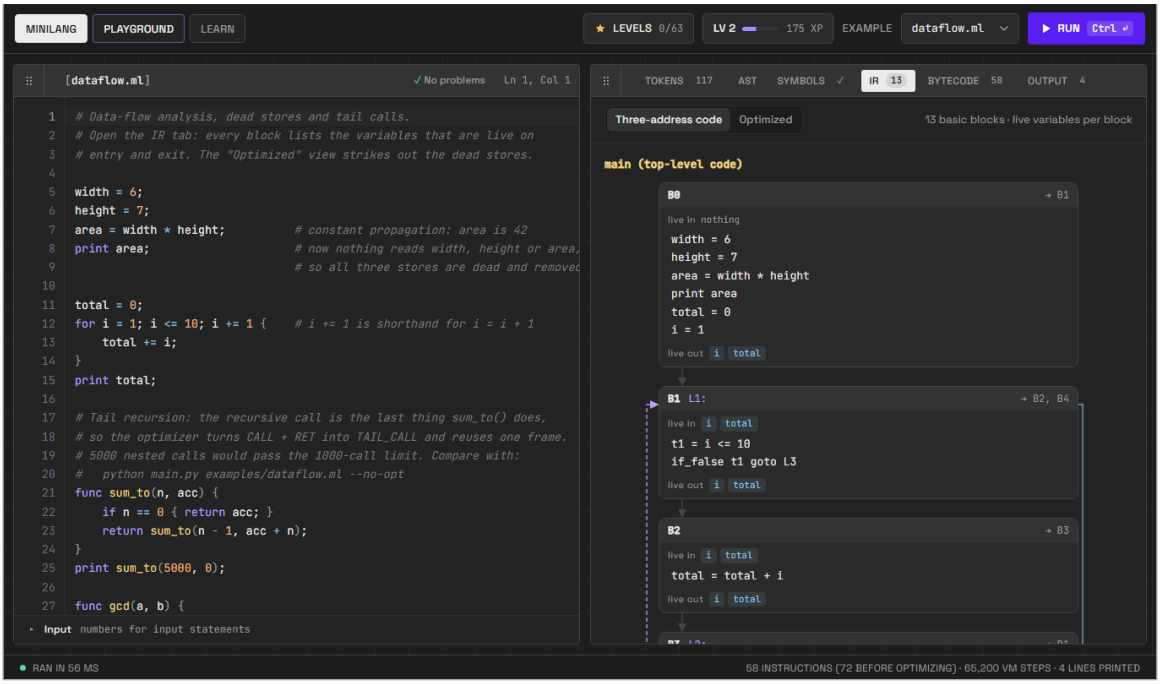


Figure 6: Playground, IR tab: basic blocks of three-address code joined into a control-flow graph, with the live variables on entry and exit of each block.

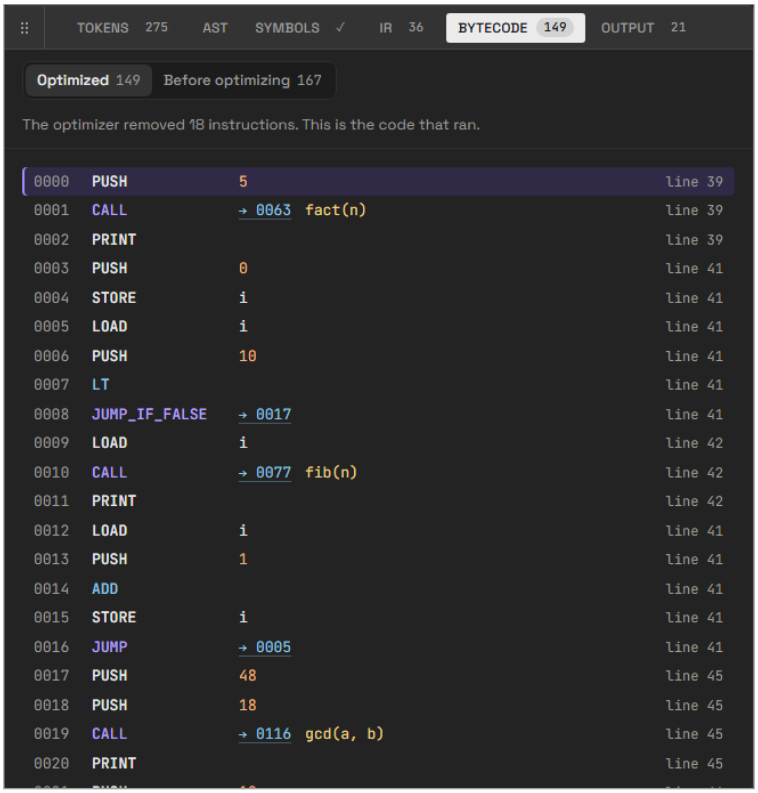


Figure 7: Playground, Bytecode tab: the optimised bytecode, with jump and call targets; a toggle shows the code before optimisation.

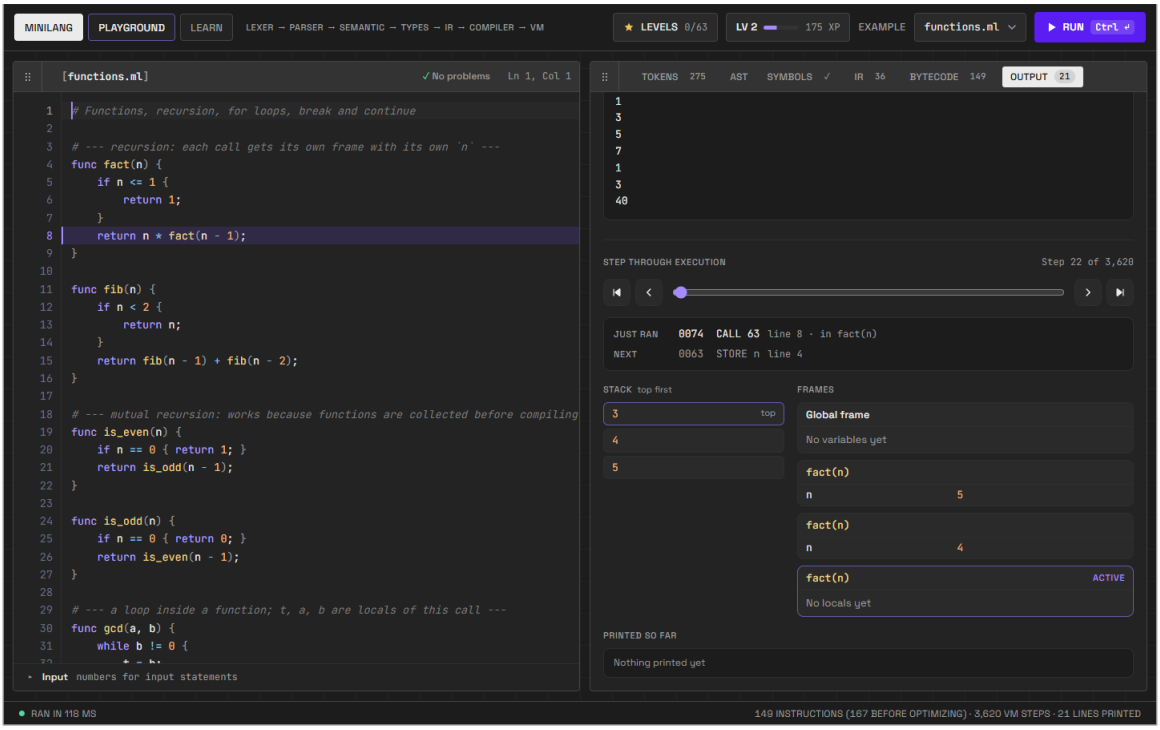


Figure 8: Playground, Output tab: stepping through execution. The operand stack and one frame per active call of the recursive factorial are shown.

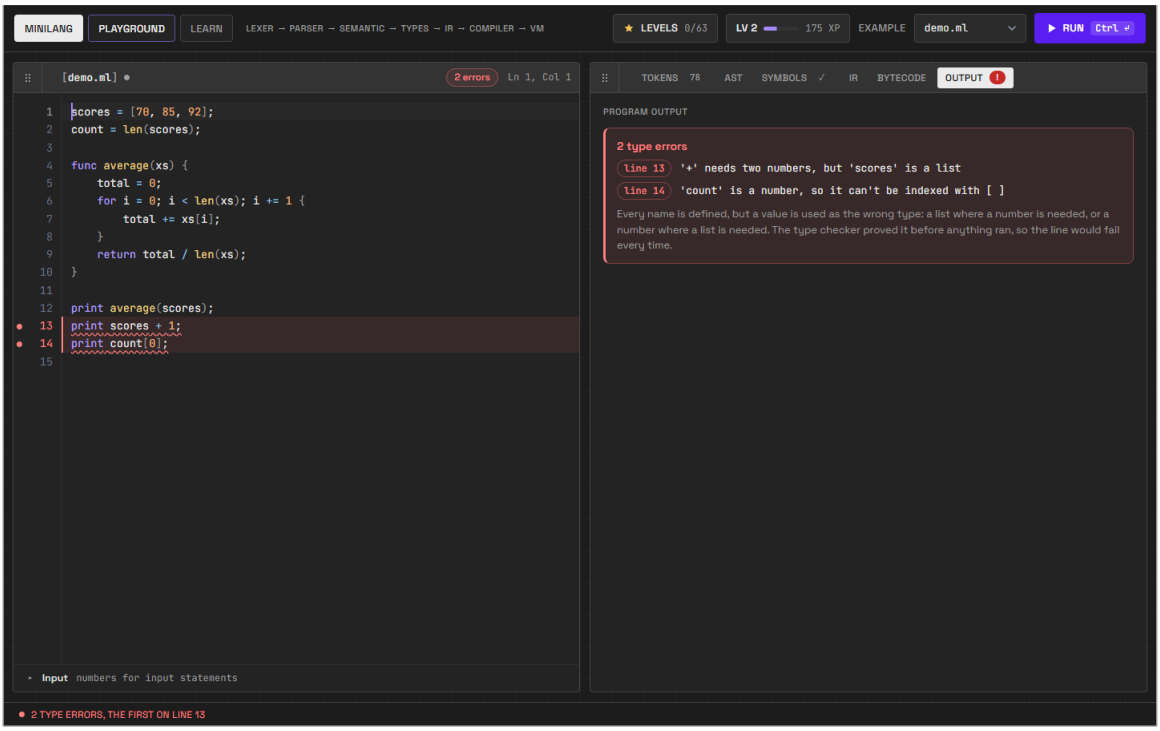


Figure 9: Live checking: type errors are underlined while typing, before the program is run.

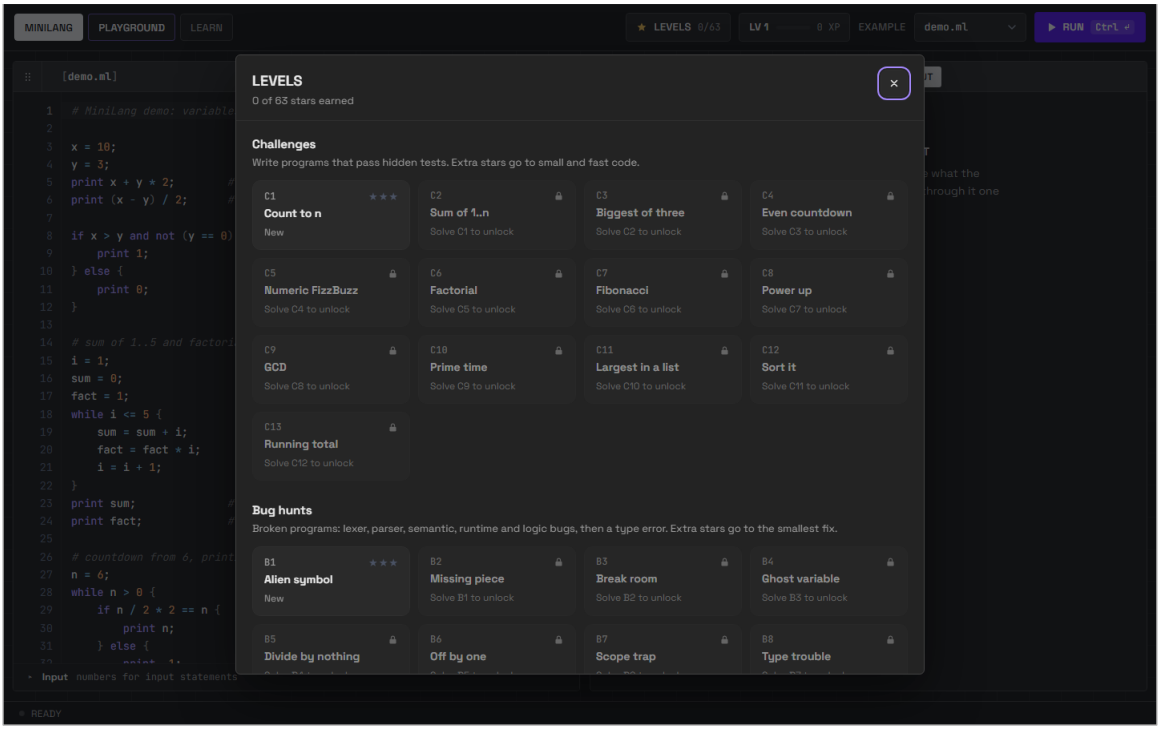


Figure 10: The level picker: 13 challenges and 8 bug hunts, each scored with up to three stars.

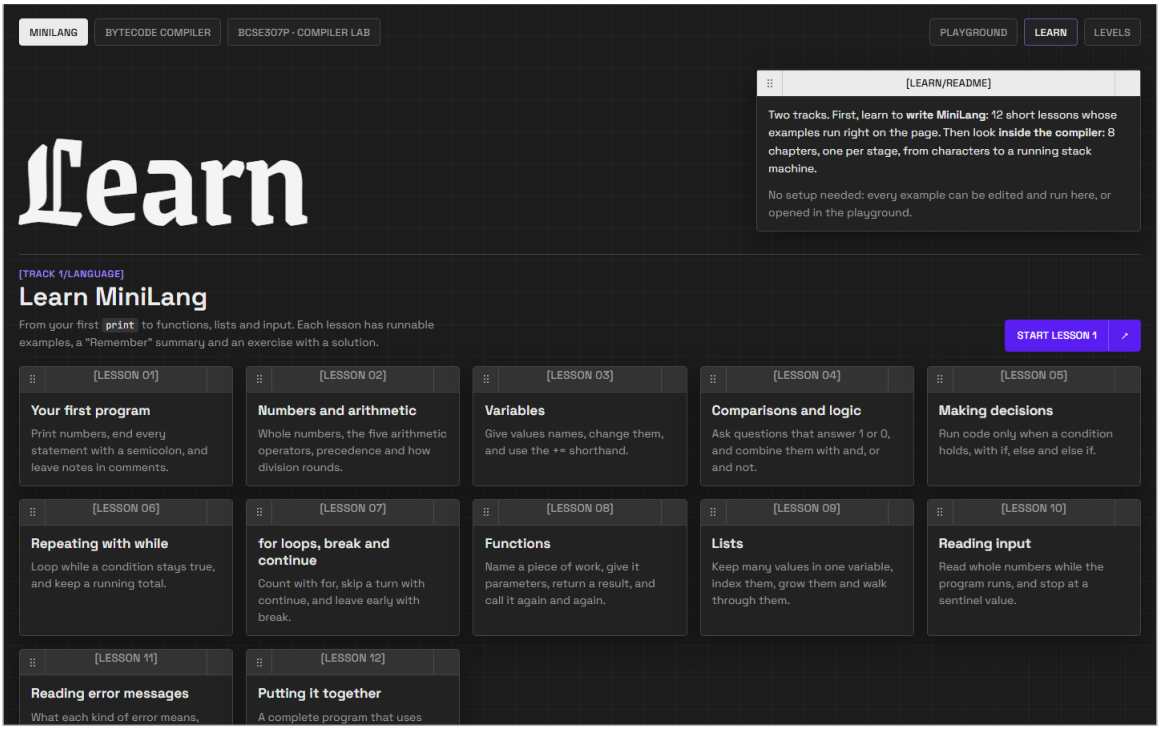


Figure 11: The learning section: two tracks, the language and the compiler.

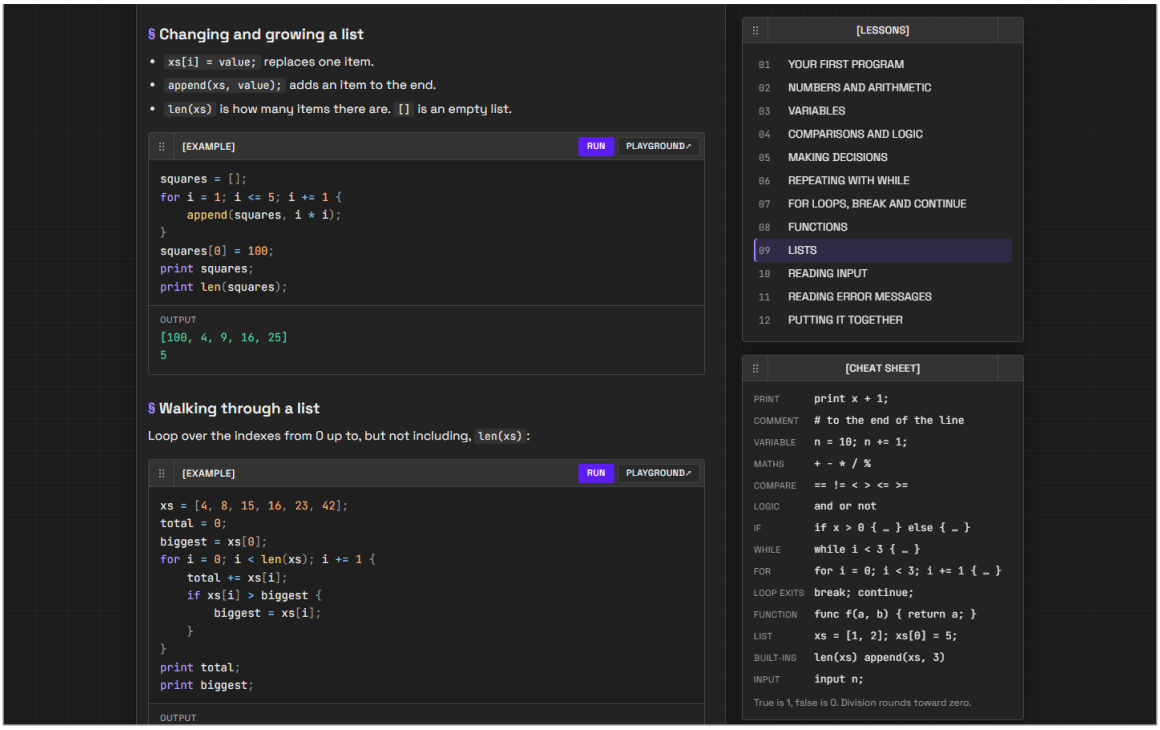


Figure 12: A language lesson. Every example is editable and runs through the real compiler.

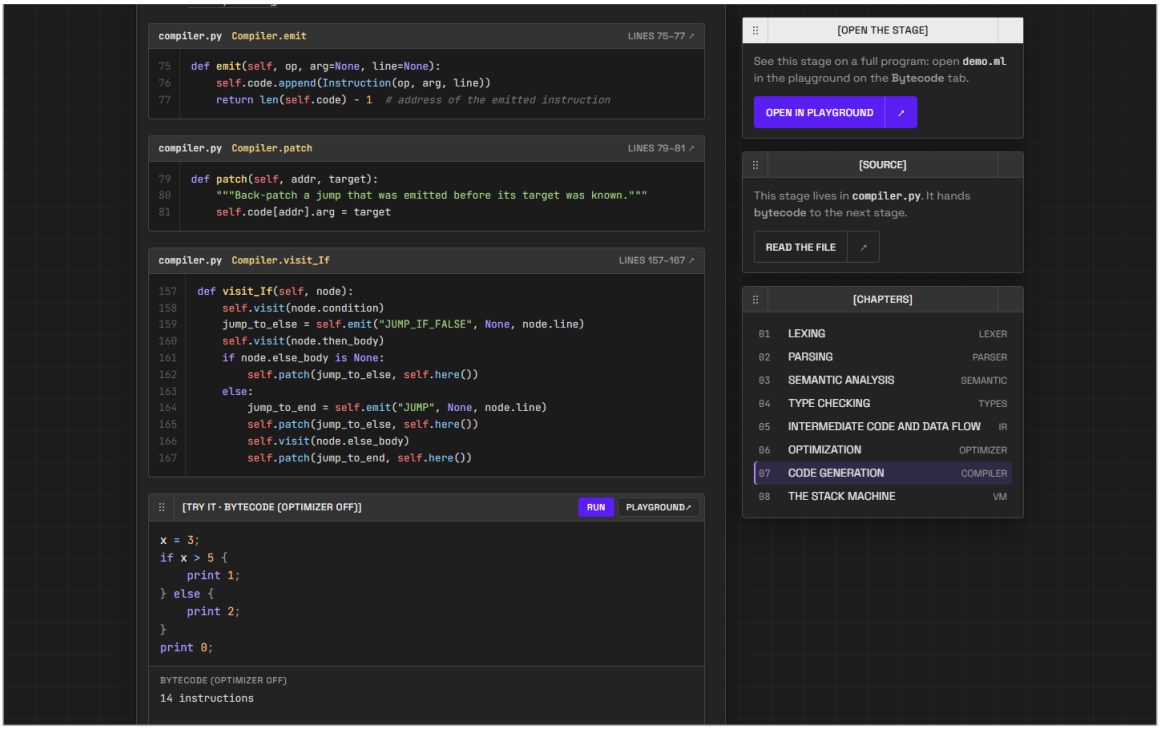


Figure 13: A compiler chapter, quoting the code generator's source directly from compiler.py.

5.7 Testing results

The test suite has 339 tests in 16 modules (Table 7). All of them pass, in about four seconds, both locally and on GitHub Actions.

Table 7: Test modules.

Test module	Tests	What it covers
`test_lexer`, `test_parser`	24	Tokens, longest match, precedence, associativity, syntax errors
`test_recovery`	28	Every lexer, parser and semantic error found in one run; hints
`test_semantic`	25	Symbol tables, scopes, warnings, the scope trap
`test_typecheck`	22	Inferred types; errors only when certainly wrong
`test_ir`	23	Three-address code, basic blocks, CFG, constants, liveness
`test_optimizer`	32	Each optimisation; same behaviour with the optimizer on and off
`test_compiler`, `test_vm`	44	Bytecode, back-patching, runtime errors and limits
`test_arrays`, `test_input`	37	Lists and the `%` operator; the `input` statement
`test_api`, `test_app`	54	JSON pipeline, web routes, offline assets, error handling
`test_levels`	20	Every reference solution earns 3 stars; every starter fails as intended
`test_guide`, `test_chapters`	30	Every lesson example and chapter quote and demo is correct

Listing 11: Running the test suite.

```
> python -m unittest discover -s tests -t .
----------------------------------------------------------------------
Ran 339 tests in 3.706s

OK
```

6. Conclusion:

MiniLang shows that a complete compiler, from characters to execution, can be built from scratch in a few thousand lines of readable code, and that every phase can be made visible. The project implements all the classical phases: lexical analysis, recursive-descent parsing, semantic analysis with symbol tables, static type inference, three-address code with basic blocks and a control-flow graph, constant propagation, liveness and definite-assignment analysis, constant folding, dead-store elimination, peephole and tail-call optimisation, code generation with back-patching, and a stack virtual machine. All of it is verified by 339 automated tests.

Building it taught us how strongly the phases depend on each other. The grammar had to be designed for one-token lookahead before the parser could be simple; the line numbers stored by the lexer are what make every later error message useful; and the optimizer is only safe because the data-flow analyses beneath it are sound. The web application, the

levels and the learning section turn the compiler into a tool for teaching these ideas, which was the project's main aim.

6.1 Limitations

- MiniLang has only whole numbers and lists: there are no strings, floating-point numbers or records.
- The target is bytecode for our own virtual machine, not native machine code, so there is no register allocation or instruction selection.
- The optimizer works on one function at a time and does not yet move code out of loops.

6.2 Future work

- Add strings, floating-point numbers and records to the language and the type checker.
- Generate native code (for example x86-64, or LLVM IR) with register allocation by graph colouring.
- Convert the intermediate code to static single assignment (SSA) form and add loop-invariant code motion and common-subexpression elimination.
- Provide editor support through the Language Server Protocol, reusing the existing error recovery.

7. Project Repository Link: (GitHub or drive link)

GitHub: https://github.com/ir1101/ByteCode

The repository contains the complete source code, the test suite, example programs and a README that documents every phase. To run the project:

Listing 12: Running MiniLang.

```
python -m pip install flask                 # the only dependency, for the website
python app.py                               # website on http://127.0.0.1:5000
python main.py examples/demo.ml --debug     # every phase on the command line
python -m unittest discover -s tests -t .   # run the 339 tests
```